

A BEGINNER'S MANUAL

# Autonomous AI Trading

Build, test, and run a hands-off trading bot, step by step

For complete beginners • October 2026 edition • Print edition · US Letter

GUIDES FOR EVERYTHING

# Autonomous AI Trading

Build, test, and run a hands-off trading bot, step by step

This manual walks you, one small step at a time, from "I have never written code" to a **trading bot that runs by itself**: it wakes up on a schedule, reads market data, decides, checks its own safety rules, places orders through a broker's API, writes a log, and sends you a daily report. You'll build it in Python with an AI assistant (Claude Code) as your pair programmer, test it on years of history, and run it on **paper money** for weeks before a single real dollar is involved.

It is also an honest book. Most people who trade actively lose money, and no bot, AI model, or course can guarantee a profit. You'll see the real studies, learn the legal and tax basics, and build the guardrails (position limits, stop-losses, daily loss limits and a kill switch) that keep a mistake small. The goal is a calm, well-tested system you understand, sized so that even its worst day is one you can shrug off.

### HOW TO READ THIS MANUAL

- **Tip** — a shortcut or good habit.
- **Note** — useful background.
- **Warning** — easy to get wrong.
- **Danger** — can lose data, money, or access.
- `$ command` — text you type, exactly as shown.
- Settings › Privacy — a menu path to click.

New words appear in **bold** the first time and are collected in the Glossary at the back.

This manual is education, not financial, legal, or tax advice. Trading involves risk, including the loss of all money you put in; past and simulated results do not predict future returns. Only trade money you can afford to lose, and paper-trade first. Broker features, fees, rules, and tax law were checked against official sources in October 2026 and change often; confirm current details with your broker, the IRS, FINRA, the SEC, or a qualified professional before acting.

# Contents


01 **Setting Up Your Computer** 4

1.1 Install Python {{Free}} 4

1.2 Install VS Code {{Free}} 5

1.3 Create your project 5

1.4 Use Git from day one 6

1.5 Install Claude Code 7

1.6 Teach Claude about your project 7

1.7 Working with AI safely 8

02 **API Key Safety** 9

2.1 Create your paper keys 9

2.2 The principle of least privilege 10

2.3 IP allowlists 10

2.4 Store keys in a .env file 11

2.5 Rules for keys 12

2.6 If a key leaks 12

**Glossary** 13

CHAPTER 1

# Setting Up Your Computer

Time to build your workshop. You'll install three free tools: **Python** (the programming language your bot is written in), **VS Code** (an editor for writing code), and **Claude Code** (an AI assistant that writes, explains, and reviews code with you).

IN THIS CHAPTER

- Installing Python 3 and VS Code
- Creating a project folder and a virtual environment
- Installing Claude Code and teaching it about your project
- Prompts for using AI as a careful coding partner

## 1.1 Install Python FREE

INSTALL PYTHON

1. **Download it.**
   Go to python.org/downloads and download the latest Python 3 release for your system. Use Python **3.12 or newer**: the current versions of pandas and vectorbt need at least 3.11.
2. **Windows: tick "Add python.exe to PATH".**
   This box is on the first installer screen. It lets your terminal find Python.
3. **Mac: run the installer package.**
   Then open **Terminal** (press Cmd + Space and type Terminal).
4. **Check it worked.**
   Open a new terminal window and type `python3 --version` (on Windows, `python --version`). You should see a version number.

A **terminal** is a window where you type commands instead of clicking. You'll only need a handful of commands, and each is explained.

## 1.2 Install VS Code FREE

Download Visual Studio Code from code.visualstudio.com and install it. Then open it, click the **Extensions** icon in the left bar, search for **Python**, and install the extension published by Microsoft. It adds colour, error hints, and a way to run scripts.

## 1.3 Create your project

### CREATE THE PROJECT FOLDER

1. **Make a folder.**
   In your terminal:

   TERMINAL
   ```
   mkdir trading-bot
   cd trading-bot
   ```

2. **Create a virtual environment.**
   A **virtual environment** is a private box of Python libraries for this one project, so it can't clash with anything else on your computer.

   TERMINAL
   ```
   python3 -m venv .venv
   ```

3. **Turn it on (activate it).**
   On Mac or Linux:

   TERMINAL
   ```
   source .venv/bin/activate
   ```

   On Windows PowerShell:

   POWERSHELL
   ```
   .venv\Scripts\Activate.ps1
   ```

   Your prompt now starts with `(.venv)`. Do this every time you open a new terminal for the project.

4 **Install the libraries.**

```
TERMINAL
pip install alpaca-py pandas python-dotenv requests pytest
```

5 **Open the folder in VS Code.**
Use File › Open Folder and pick `trading-bot`. When VS Code asks which Python to use, choose the one inside `.venv`.

Here's what each library does:

| LIBRARY | JOB |
|---|---|
| `alpaca-py` | Talks to Alpaca: prices, account, orders |
| `pandas` | Tables of prices, averages, and statistics |
| `python-dotenv` | Loads secret keys from a `.env` file |
| `requests` | Sends alert messages over the internet |
| `pytest` | Runs automatic tests of your code |

Save the list so you (or a server) can reinstall it exactly:

```
TERMINAL
pip freeze > requirements.txt
```

## 1.4 Use Git from day one

**Git** keeps a history of every change to your code, so you can undo mistakes. Install it from git-scm.com, then in your project folder:

```
TERMINAL
git init
```

Create a file named `.gitignore` so secrets and junk never get saved into history:

```
.GITIGNORE
.venv/
.env
state.json
logs/
__pycache__/
KILL_SWITCH
```

> **DANGER**
> The line `.env` is the most important line in that file. Your `.env` file will hold API keys. If it's ever pushed to GitHub, assume the keys are stolen and delete them at the broker immediately.

## 1.5 Install Claude Code

Claude Code is an AI coding agent that works inside your project folder. It needs a paid Claude plan (Pro or higher) or an Anthropic Console account. On macOS, Linux, or WSL:

TERMINAL

```
curl -fsSL https://claude.ai/install.sh | bash
```

On Windows PowerShell:

POWERSHELL

```
irm https://claude.ai/install.ps1 | iex
```

Open a new terminal, go to your project folder, and type `claude`. Follow the login steps in your browser. (The companion manual *Claude Code* covers this in depth.)

## 1.6 Teach Claude about your project

Claude Code reads a file named `CLAUDE.md` in your project at the start of every session. Use it to set ground rules:

CLAUDE.MD

```
# Trading bot project rules
- This bot trades REAL money if misconfigured. Be careful.
- Always default to Alpaca PAPER trading (paper=True).
- Never print, log, or commit API keys. Secrets live in .env only.
- Only bot/orders.py may submit orders.
- Every order must pass bot/risk.py checks first.
- Write a pytest test for every new rule or signal.
- Explain changes in plain English for a beginner.
- Never add margin, shorting, options, or leverage.
```

Then block Claude from reading your secrets file. Create `.claude/settings.json`:

.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "deny": [
      "Read(./.env)"
    ]
  }
}
```

## 1.7 Working with AI safely

AI is a superb tutor and a fast typist, but it can be confidently wrong, especially about library details that changed recently. Use it like this:

- **Ask it to explain before it writes.** "Explain how you'll do this, then wait."
- **Make it test.** Ask for a test with every function.
- **Ask for a review.** A second pass catches many bugs.
- **Check names against the docs.** If an import fails, paste the error back and ask it to check the library's current documentation.

PROMPT

I'm a complete beginner building a Python trading bot with alpaca-py on a PAPER account. Read CLAUDE.md first. Explain, in plain English, what files a simple daily bot needs and what each one does. Don't write any code yet.

PROMPT

Review bot/risk.py like a cautious senior engineer. List every way this code could let through a trade that is too big, repeated, or on the wrong symbol. Then write pytest tests for each case.

- [ ] `python3 --version` shows 3.12 or newer
- [ ] VS Code is installed with the Python extension
- [ ] My project has a `.venv` and the libraries installed
- [ ] `.gitignore` lists `.env`
- [ ] Claude Code starts in my project and has a `CLAUDE.md`
- [ ] Claude is denied from reading `.env`

CHAPTER 2

# API Key Safety

An **API key** is a pair of secret codes that lets your bot act on your account without your password. Usually there's a **key ID** (like a username) and a **secret key** (like a password). Anyone holding both can do whatever the key is allowed to do. Treat them like the keys to your house.

IN THIS CHAPTER

- Creating paper keys at Alpaca
- Least privilege: trade-only, no withdrawals
- IP allowlists and expiry dates
- Storing keys in a `.env` file
- What to do if a key leaks

## 2.1 Create your paper keys

GENERATE ALPACA PAPER KEYS

1. **Log in and switch to your paper account.**
   Check the account switcher at the top left.
2. **Find the API Keys panel.**
   On the dashboard home page it's in the right sidebar.
3. **Click Generate New Keys.**
   Alpaca shows your key ID and secret.
4. **Copy both into your `.env` file immediately** (next section). The secret is shown only once; if you lose it, regenerate the keys.

> **NOTE**
>
> Paper and live accounts have **different keys**. A paper key can't touch real money. That's a natural safety barrier, so keep paper keys and live keys in separate files and never mix them up.

## 2.2 The principle of least privilege

**Least privilege** means giving each key only the powers it truly needs. A trading bot needs to read balances and place and cancel orders. It never needs to **withdraw** or **transfer** money.

| EXCHANGE | TURN ON | TURN OFF |
|---|---|---|
| **Kraken** | Query Funds, Query Open Orders & Trades, Modify Orders, Cancel/Close Orders | Withdraw Funds, Deposit Funds |
| **Coinbase (CDP)** | View and Trade permissions you need | Transfer |
| **Any broker** | Trading for the one account the bot uses | Anything about money movement |

Coinbase's documentation spells out why: if someone obtains an API secret with the **transfer** permission, they can send all the crypto out of your account. Without it, a thief can trade badly but can't empty the account.

## 2.3 IP allowlists

An **IP address** is your server's address on the internet. An **IP allowlist** (also called whitelisting) tells the exchange: "only accept this key from these addresses." Coinbase and Kraken both offer it, and Coinbase recommends it.

### LOCK A KEY TO YOUR SERVER

1. **Find your server's public IP.**
   On the server, run `curl -4 ifconfig.me`. (On a home connection the IP can change; a cloud server's is usually fixed.)
2. **Edit the key.**
   In the exchange's API key settings, add that address to the allowlist.
3. **Test.**
   Run the bot once from the server. Then try from your laptop: it should be refused.

Kraken also lets you give a key an **expiration date**. A key that expires after three months forces a regular, healthy rotation.

## 2.4 Store keys in a .env file

Never type keys into your code. Put them in a file called `.env` in your project folder:

.ENV

```
ALPACA_API_KEY=PKXXXXXXXXXXXXXXXXXX
ALPACA_SECRET_KEY=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=
SLACK_WEBHOOK_URL=
```

Your code loads it with `python-dotenv`:

PYTHON

```python
import os
from dotenv import load_dotenv

load_dotenv()                       # reads .env into the environment
key = os.environ["ALPACA_API_KEY"]  # crashes loudly if missing: good
```

Also create a `.env.example` with the same names and no values. That file is safe to commit and reminds you which settings exist.

> **TIP**
> On Mac or Linux, make the file readable only by you: `chmod 600 .env`.

## 2.5 Rules for keys

**Do**

- Use separate keys for paper and live
- Give live keys trade-only permissions
- Add an IP allowlist where offered
- Rotate keys every few months
- Turn on two-factor authentication on the account

**Never**

- Paste keys into code, chat apps, screenshots or AI prompts
- Commit `.env` to Git
- Give withdrawal permission to a bot
- Share a key with a “signal service” or third-party bot

## 2.6 If a key leaks

EMERGENCY RESPONSE

1. **Delete or regenerate the key at the broker.**
   Do this first, before investigating.
2. **Check recent orders and activity** in the dashboard for anything you didn’t do.
3. **Cancel unexpected open orders** and close unexpected positions.
4. **Find the leak.**
   A public GitHub repo? A pasted log? Fix it.
5. **Create a new key** with minimal permissions and update `.env`.

- [ ] I’ve generated Alpaca **paper** keys and saved them in `.env`
- [ ] `.env` is in `.gitignore`, and `.env.example` has no values
- [ ] I know which permissions a live key should never have
- [ ] I know how to add an IP allowlist and what to do after a leak

# Glossary

| | |
|---|---|
| **Adjusted prices** | Historical prices rewritten to remove the effect of stock splits and dividends. |
| **AI agent** | An AI that can take actions, such as placing orders, not just answer questions. |
| **API** | Application programming interface: a doorway that lets one program talk to another, such as your bot talking to your broker. |
| **API key** | A pair of secret codes (key ID and secret) that lets a program act on your account. |
| **Atomic write** | Saving a file by writing a temporary copy and swapping it in, so the file is never half-written. |
| **Backtest** | Replaying a strategy on historical prices to see how it would have performed. |
| **Bar** | A summary of trading over a period (open, high, low, close, volume). Also called a candle. |
| **Benchmark** | What you compare a strategy with, usually buying and holding the same fund. |
| **Bracket order** | An entry order plus a take-profit and a stop-loss that activate when it fills. |
| **Broker** | A company that holds your investment account and sends your orders to the market. |
| **CAGR** | Compound annual growth rate: the average yearly return, with compounding. |
| **Cash account** | A brokerage account where you can only buy with deposited money; no borrowing. |
| **Circuit breaker** | A rule that halts the bot when results suggest something is wrong, such as a large drawdown. |

| | |
|---|---|
| **Client order ID** | Your own label attached to an order, so you can trace it back to a specific run. |
| **cron** | The classic Linux scheduler that runs commands at set times. |
| **Crypto exchange** | A company that runs a marketplace for cryptocurrencies, usually holding your coins. |
| **CSV** | Comma-separated values: a plain-text table that opens in any spreadsheet. |
| **Daily loss limit** | A rule that stops trading for the day after the account falls a set percentage. |
| **Dataclass** | A Python container for a group of related settings or values. |
| **DataFrame** | A pandas table of rows and columns, used for prices and calculations. |
| **Day trading** | Opening and closing positions within the same trading day. |
| **Digital assets question** | The yes/no question about crypto on U.S. federal tax returns. |
| **Digital commodity** | A crypto asset, such as bitcoin, that the SEC and CFTC treat as a commodity rather than a security. |
| **Dividend** | A cash payment a company or fund makes to its shareholders. |
| **Dollar-cost averaging (DCA)** | Buying a fixed dollar amount at regular intervals regardless of price. |
| **Drawdown** | The fall from an account's or strategy's highest value to a later low. |
| **Dry run** | Running the bot so it makes and logs decisions without sending any orders. |
| **Equity curve** | A chart or series showing how an account's value changed over time. |
| **ETF** | Exchange-traded fund: a basket of investments that trades like a single stock. |
| **Form 1099-DA** | The IRS form crypto brokers use to report your digital asset sales. |
| **Fractional share** | Part of one share, bought by specifying a dollar amount. |
| **Free-riding** | Buying and selling a security in a cash account before paying for it; can lead to a 90-day restriction. |
| **Good faith violation** | Selling a security bought with unsettled funds before those funds settle, in a cash account. |
| **Grid bot** | A bot that places buy and sell orders at evenly spaced prices to profit from price swings in a range. |
| **GTC** | Good-till-cancelled: an order that stays open until filled or cancelled (Alpaca cancels them after 90 days). |

| | |
|---|---|
| **Guardrail** | A rule that keeps a component's mistakes harmless, such as defaulting an unexpected AI answer to "neutral." |
| **Heartbeat** | A check that the bot actually ran, so silent failures are noticed. |
| **IEX** | Investors Exchange, a single U.S. stock exchange whose real-time data Alpaca provides free. |
| **Intraday margin standards** | FINRA's 2026 replacement for the pattern day trader rules, based on exposure during the day. |
| **IP allowlist** | A setting that only accepts an API key from listed internet addresses. Also called whitelisting. |
| **Kill switch** | A way to stop the bot and close all positions immediately. |
| **Large language model (LLM)** | An AI model that reads and writes text, such as Claude. |
| **Least privilege** | Giving each key or tool only the permissions it truly needs. |
| **Leverage** | Controlling a larger position than your deposit, magnifying gains and losses. |
| **Liquidity** | How easily something can be bought or sold without moving its price. |
| **Look-ahead bias** | A backtest error that uses information you wouldn't have had at the time. |
| **Machine learning (ML)** | Letting a program find patterns in data instead of writing rules by hand. |
| **Maker / taker** | A maker order waits on the order book and adds liquidity; a taker order fills immediately. Fees often differ. |
| **Margin account** | A brokerage account that lets you borrow to buy or sell short. |
| **Market data** | Prices and volumes coming from exchanges. |
| **Market order** | An order to buy or sell immediately at the best available price. |
| **MCP** | Model Context Protocol: a standard way to connect AI apps to outside tools such as a broker. |
| **Mean reversion** | A strategy betting that unusually stretched prices return toward their average. |
| **Momentum** | A strategy that buys recent relative winners, expecting them to keep outperforming for a while. |
| **Moving average** | The average price over the last N days, recalculated daily. |
| **Notional order** | An order specified in dollars rather than shares. |

| | |
|---|---|
| **OHLCV** | Open, high, low, close, volume: the five numbers in a price bar. |
| **Order** | An instruction to your broker to buy or sell. |
| **OTO order** | One-triggers-other: an entry order that activates a second order (such as a stop-loss) when it fills. |
| **Overfitting** | Tuning rules so closely to past data that they capture noise and fail on new data. |
| **Paper trading** | Simulated trading with fake money against real prices. |
| **Pattern day trader (PDT)** | A former FINRA designation for frequent day traders in margin accounts, with a $25,000 minimum; replaced in 2026. |
| **Position** | The amount of a particular investment you currently hold. |
| **Position sizing** | Deciding how much to buy, for example with the 1–2% risk rule. |
| **Prompt injection** | Hidden text that tries to trick an AI into doing something it shouldn't. |
| **Pure function** | A function that only uses its inputs and returns an answer, with no network or file access. |
| **Rebalancing** | Trading back to a target mix of investments when it drifts. |
| **Risk rules** | Code that can veto a trade, such as size limits and loss limits. |
| **Rotating log** | A log file that starts a new file at a size limit and keeps only a few old ones. |
| **Sandbox** | A test version of an API; some simulate trading, others return canned answers. |
| **Settlement** | When a trade officially completes and cash or shares change hands; T+1 for most U.S. stocks. |
| **Sharpe ratio** | Return divided by volatility; a measure of return per unit of risk. |
| **Short selling** | Selling borrowed shares to profit if the price falls; requires margin. |
| **Short-term capital gain** | Profit on an asset held one year or less; taxed as ordinary income in the U.S. |
| **SIP** | Securities information processor: the combined data feed of all U.S. exchanges. |
| **SIPC** | Securities Investor Protection Corporation: protects customers if a member brokerage fails, up to $500,000 including $250,000 cash. |
| **Slippage** | The difference between the price you expected and the price you got. |
| **Spot** | Buying the actual asset (such as a coin) rather than a contract about it. |

| | |
|---|---|
| **Spread** | The gap between the best buying and selling prices. |
| **Standard deviation** | A measure of how spread out values are around their average. |
| **State file** | A small file where the bot remembers things between runs. |
| **Stop-loss** | An order that sells automatically if the price falls to a chosen level. |
| **Strategy** | Written rules precise enough for a computer to follow. |
| **Survivorship bias** | Testing only on assets that still exist, ignoring those that failed. |
| **systemd timer** | A Linux scheduler built into Ubuntu that can catch up on missed runs. |
| **T+1** | Settlement one business day after the trade date. |
| **Terminal** | A window for typing commands instead of clicking. |
| **Timeframe** | How often the bot looks at prices and can trade (daily, hourly, and so on). |
| **TOML** | A simple settings file format of names and values. |
| **Trend following** | A strategy that holds assets while their prices are trending up. |
| **Virtual environment** | A private set of Python libraries for one project. |
| **VM** | Virtual machine: a rented computer in a data centre. |
| **Walk-forward testing** | Choosing parameters on past data, testing them on the next unseen period, and rolling forward. |
| **Wash sale** | Selling at a loss and buying substantially identical securities within 30 days before or after; the loss is deferred. |
| **Watchlist** | The short list of symbols the bot is allowed to trade. |
| **Z-score** | How many standard deviations a value is from its average. |